

Xu Wang

Prof. Simon Shim

Data 260

31 August 2026

DATA 260 — Homework 1

Repository: <https://github.com/xuwang-goldenwater/data260-9275>

Section 0 — Personal Configuration and Domain

All values below are derived from the last four digits of my SJSU Student ID and stay fixed for the rest of the semester.

SID4	9275
PORT_BASE	8275 (8000 + 9275 mod 900)
PREFIX	s9275
SEED	9275
VERIFY_SEED	269275 (260000 + 9275)
DOMAIN_ID	3 (9275 mod 8) — Grocery supply and recall notices

Environment

Hardware	MacBook Pro (Apple silicon), 8 GB RAM, macOS
Local model	qwen3:8b via Ollama (Q4_K_M, context length 40,960)
Python	3.12 in the conda environment `data260`
Container runtime	Docker Desktop, image built for linux/amd64
Cloud	AWS ECS Fargate, region us-east-1, account 457145067527

Tagged commit hash (tag hw1): ca3b733

Domain entity

My assigned domain is grocery supply and recall notices. The entity modeled throughout this homework is a RecallNotice: a public announcement that a food product already on the market is being pulled back because of a safety or labeling problem. The full field definition is in DOMAIN_SCHEMA.md at the repository root.

Part 1 — Section I: HTML

File: code/web_application/index.html

I.1 — Create an HTML page titled HW1-{Your Name}

The `<title>` element sets the text shown on the browser tab.

```
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <!-- Requirement I.1: page title -->
  <title>HW1-Xu Wang</title>
  ...
</head>
```

I.2 — Heading tag naming the assigned domain entity

HTML heading tags run from `<h1>` to `<h6>`. `<h1>` renders at the largest font size, so it is the tag required here.

```
<!-- Requirement I.2: largest heading tag naming the domain entity -->
<h1>Recall Notice</h1>
```

Outputs of I.1 and I.2

HW1-Xu Wang x FA26: DATA-260 Sec 21 & x New T

s/data260-9275/code/web_application/index.html

Recall Notice

Product Name *

Recalling Firm *

Submitter Email *

Recall Details *

Reason for Recall *

☐ I agree to the terms and conditions.

I.3 — DOMAIN_SCHEMA.md defining the entity fields and category values

DOMAIN_SCHEMA.md was written before any code. It fixes the seven field names so that the HTML id attributes, the JavaScript keys, and the JSON sent to the agent in Part 2 all match.

Fields

productName	string, required — name of the recalled product. Used as the agent title.
recallingFirm	string, required — company issuing the recall.
submitterEmail	email, required — contact for follow-up.
description	string, required — what is wrong, affected lots, consumer action. Used as the agent content.
category	enum, required — reason for the recall.
agreedToTerms	boolean, required — must be true.
submissionDate	ISO 8601 string, derived — added in JavaScript at submission time.

Category values

Undeclared Allergen | Bacterial Contamination | Foreign Material | Mislabeling

These are the three most common real causes of FDA food recalls plus labeling error. In Q1 2026, undeclared allergens accounted for 40.7% of FDA food recalls, foreign material 17.1%, and bacterial contamination 15.7%. Severity class (Class I / II / III) is assigned by the regulator after review rather than by the submitter, so it is not part of this form.

For more details, please go to the DOMAIN_SCHEMA.md file.

I.4 — Form for the domain entity

Each of the seven requirements a–g maps to one control.

I.4.a — Required text input for the primary field, with placeholder and autofocus

```
<label for="productName">Product Name *</label>
<input
  type="text"
  id="productName"
  placeholder="Recalled product name, e.g. Creamy Peanut Butter, 16 oz jar"
  required
  autofocus
/>
```

required blocks an empty submit; autofocus puts the cursor in this field as soon as the page loads.

I.4.b — Required text input for the secondary field, with placeholder

```
<label for="recallingFirm">Recalling Firm *</label>
<input
  type="text"
  id="recallingFirm"
  placeholder="Company issuing the recall, e.g. Sunrise Valley Foods, Inc."
  required
/>
```

I.4.c — Required email input for the submitter's email

```
<label for="submitterEmail">Submitter Email *</label>
<input
  type="email"
  id="submitterEmail"
  placeholder="Contact email for follow-up, e.g. quality@example.com"
  required
/>
```

type="email" makes the browser check the address format before the form is submitted.

I.4.d — Required text area for the content / description field

```
<label for="description">Recall Details *</label>
<textarea
  id="description"
  placeholder="Describe the problem, affected lot codes and best-by dates, distribution area,
  and what consumers should do"
  required
></textarea>
```

I.4.e — Dropdown for category selection with four domain-appropriate options

```

<label for="category">Reason for Recall *</label>
<select id="category" required>
  <option value="" disabled selected>Select a reason</option>
  <option value="Undeclared Allergen">Undeclared Allergen</option>
  <option value="Bacterial Contamination">Bacterial Contamination</option>
  <option value="Foreign Material">Foreign Material</option>
  <option value="Mislabeling">Mislabeling</option>
</select>

```

The first option is a placeholder: disabled so it cannot be chosen, selected so it shows by default, and value="" so required rejects it.

I.4.f — Checkbox and label reading “I agree to the terms and conditions.”

```

<div class="terms">
  <input type="checkbox" id="agreedToTerms" />
  <label for="agreedToTerms">I agree to the terms and conditions.</label>
</div>

```

The checkbox deliberately has no required attribute. Requirement II.1.b asks for this check to be done in JavaScript with an alert; a required attribute would let the browser block the submit first and the JavaScript branch would never run.

I.4.g — Submit button with domain-appropriate text

```

<button type="submit">Submit Recall Notice</button>

```

Output — complete form & required-field validation

The screenshot shows a web browser window with the URL `/Users/Sunny/Documents/data260-9275/code/web_application/index.html`. The page displays a "Recall Notice" form. The form has the following sections:

- Product Name ***: A text input field with a red border and a placeholder text "Recalled product name, e.g. Creamy Peanut Butter, 16 oz jar".
- Recalling Firm ***: A text input field with a red border and a placeholder text "Company issuing the recall, e.g. Sunrise Valley Foods, Inc.". A red error message "Please fill out this field." is displayed above the field.
- Submitter Email ***: A text input field with a placeholder text "Contact email for follow-up, e.g. quality@example.com".
- Recall Details ***: A text area with a placeholder text "Describe the problem, affected lot codes and best-by dates, distribution area, and what consumers should do".
- Reason for Recall ***: A dropdown menu with a placeholder text "Select a reason".
- Terms and Conditions**: A checkbox labeled "I agree to the terms and conditions.".
- Submit Button**: A red button labeled "Submit Recall Notice".

The browser's developer console is open on the right, showing a message: "cmd to turn on code suggestions. Don't show again NEW".

Output — category dropdown

The screenshot shows a web browser with two tabs labeled 'HW1-Xu Wang'. The address bar shows a file path: `/Users/Sunny/Documents/data260-9275/code/web_application/i`. The main content is a form titled 'Recall Notice'.

The form contains the following fields:

- Product Name ***: A text input field with placeholder text 'Recalled product name, e.g. Creamy Peanut Butter, 16 oz jar'.
- Recalling Firm ***: A text input field with placeholder text 'Company issuing the recall, e.g. Sunrise Valley Foods, Inc.'.
- Submitter Email ***: A text input field with placeholder text 'Contact email for follow-up, e.g. quality@example.com'.
- Recall Details ***: A text area with placeholder text 'Describe the problem, affected lot codes and best-by dates, distribution area, and what consumers should do'.
- Reason for Recall ***: A dropdown menu with a checkmark icon. The menu is open, showing four options: 'Undeclared Allergen', 'Bacterial Contamination', 'Foreign Material', and 'Mislabeling'.

I.5 — Script tag linking the Part II JavaScript at the end of the file

```
<!-- Requirement I.5: link the Part II JavaScript at the end of the file -->
<script src="app.js"></script>

</body>
</html>
```

The browser parses the document from top to bottom. The script looks up the form by its id, so it has to run after the form element exists. Placing the tag just before `</body>` guarantees that; a tag in `<head>` would run too early and find nothing.

Part 1 — Section II: JavaScript

File: code/web_application/app.js

All console output below is produced by the submit handler registered on the form. The handler calls `event.preventDefault()` so the page does not reload and the console output stays visible.

II.1 — Arrow function validating the form

II.1.a the description must be longer than 25 characters; II.1.b the terms checkbox must be checked. Each failure shows an alert and stops the submission.

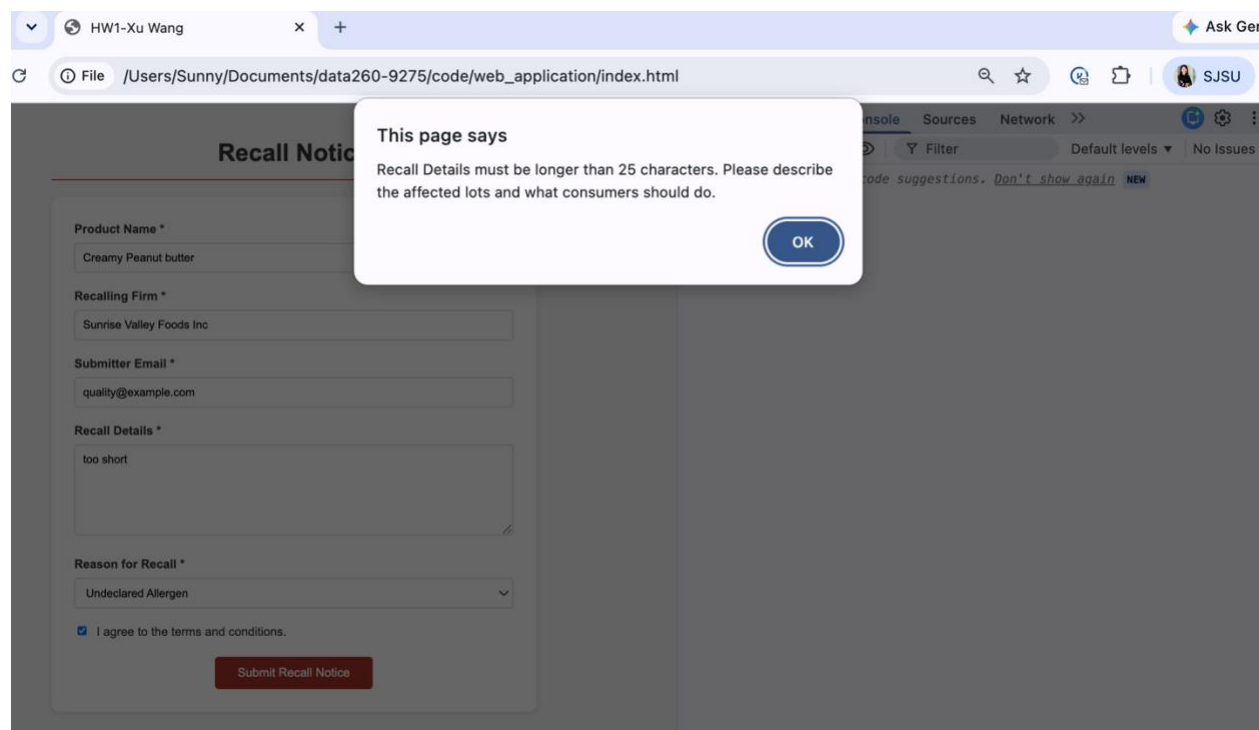
```
const validateRecallNotice = (description, agreedToTerms) => {
  if (description.trim().length <= 25) {
    alert(
      "Recall Details must be longer than 25 characters. " +
      "Please describe the affected lots and what consumers should do."
    );
    return false;
  }

  if (!agreedToTerms) {
    alert("Please agree to the terms and conditions before submitting this recall notice.");
    return false;
  }

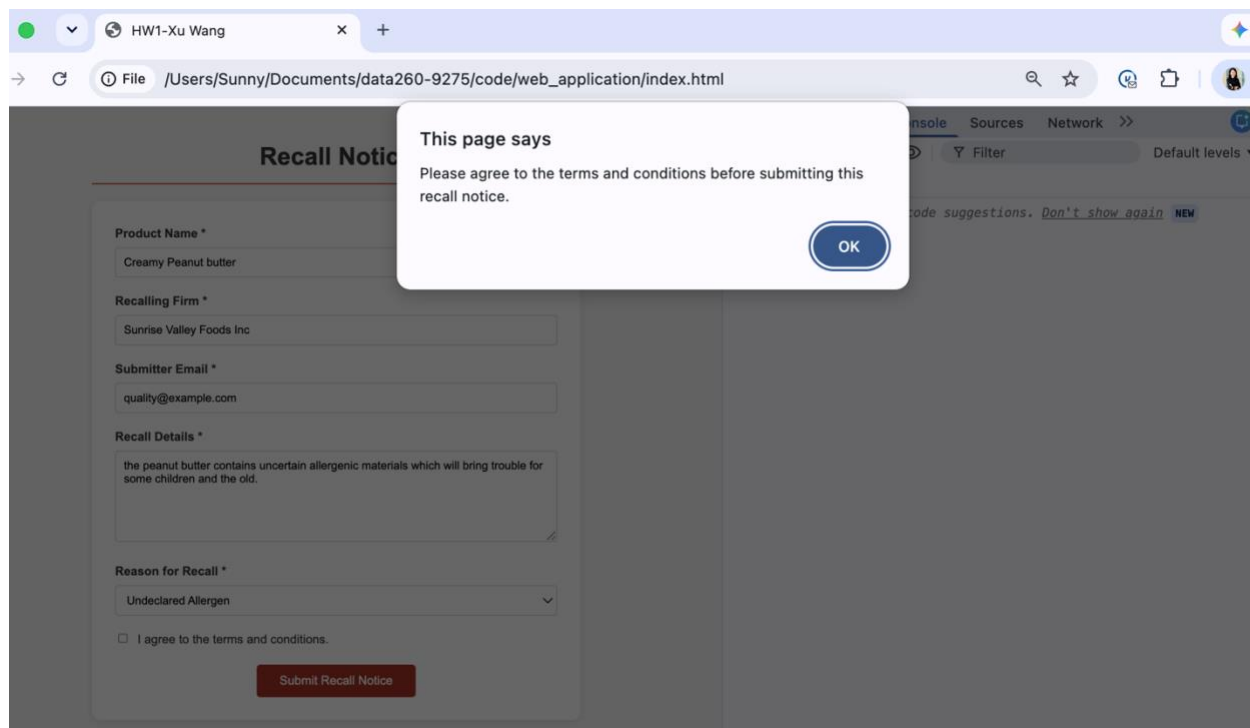
  return true;
};
```

An arrow function is written as `(parameters) => { body }`. It is a shorter way of writing `function(parameters) { body }`.

Output — II.1.a short description



Output — II.1.b terms not accepted



II.2 — Convert the form data into a JSON string and log it

```
const formData = {
  productName: document.getElementById("productName").value,
  recallingFirm: document.getElementById("recallingFirm").value,
  submitterEmail: document.getElementById("submitterEmail").value,
  description: document.getElementById("description").value,
  category: document.getElementById("category").value,
  agreedToTerms: document.getElementById("agreedToTerms").checked,
};

const jsonString = JSON.stringify(formData, null, 2);
console.log("II.2 Form data as a JSON string:");
console.log(jsonString);
```

JSON.stringify turns a JavaScript object into text. The third argument, 2, indents the output by two spaces so it is readable in the console.

II.3 — Object destructuring for the primary field and email

```
const parsedData = JSON.parse(jsonString);

const { productName, submitterEmail } = parsedData;
console.log("II.3 Destructured from the parsed object:");
console.log("    productName    :", productName);
console.log("    submitterEmail:", submitterEmail);
```

Destructuring pulls named properties out of an object into standalone variables in one statement, instead of writing `parsedData.productName` and `parsedData.submitterEmail` separately. The names inside the braces must match the property names.

II.4 — Spread operator adding submissionDate

```
const parsedDataWithDate = {
  ...parsedData,
  submissionDate: new Date().toISOString(),
};
console.log("II.4 Parsed object with submissionDate added:");
console.log(parsedDataWithDate);
```

The spread operator `...` copies every property of `parsedData` into a new object, and `submissionDate` is then added to that copy. The original `parsedData` is left unchanged.

Output of II.2, II.3, and II.4

The screenshot displays a web browser window with a single tab titled 'HW1-Xu Wang'. The address bar shows the file path: `/Users/Sunny/Documents/data260-9275/code/web_application/index.html`. The browser's developer tools are open, showing the 'Console' tab with several log messages.

Recall Notice Form:

- Product Name ***: Text input with placeholder text "Recalled product name, e.g. Creamy Peanut Butter, 16 oz jar".
- Recalling Firm ***: Text input with placeholder text "Company issuing the recall, e.g. Sunrise Valley Foods, Inc.".
- Submitter Email ***: Text input with placeholder text "Contact email for follow-up, e.g. quality@example.com".
- Recall Details ***: Text area with placeholder text "Describe the problem, affected lot codes and best-by dates, distribution area, and what consumers should do".
- Reason for Recall ***: A dropdown menu with the selected option "Select a reason".
- ☐ I agree to the terms and conditions.
- Submit Recall Notice**: A red button.

Console Output:

```

=== Recall notice submitted successfully === app.js:69
II.2 Form data as a JSON string: app.js:73
{ app.js:74
  "productName": "Oranges",
  "recallingFirm": "Taylor Inc",
  "submitterEmail": "fruit@example.com",
  "description": "The oranges sent to some markets was mislabeled with
organic, while the test results show that there are chemicals on the
surface of the oranges.",
  "category": "Mislabeling",
  "agreedToTerms": true
}
II.3 Destructured from the parsed object: app.js:81
  productName : Oranges app.js:82
  submitterEmail: fruit@example.com app.js:83
II.4 Parsed object with submissionDate added: app.js:90
app.js:91
{productName: 'Oranges', recallingFirm: 'Taylor Inc', submitterEmail: 'f
ruit@example.com', description: 'The oranges sent to some markets was mi
s-labeled wi- here are chemicals on the surface of the oranges.', categor
y: 'Mislabeling', ...} 1
  agreedToTerms: true
  category: "Mislabeling"
  description: "The oranges sent to some markets was mislabeled with orga
productName: "Oranges"
recallingFirm: "Taylor Inc"
submissionDate: "2026-08-31T17:41:14.630Z"
submitterEmail: "fruit@example.com"
  [[Prototype]]: Object
II.5 Successful submission count: 1 app.js:95

```

II.5 — Closure tracking the number of successful submissions

```
const createSubmissionCounter = () => {
  let count = 0;
  return () => {
    count = count + 1;
    return count;
  };
};

const countSubmission = createSubmissionCounter();

// inside the submit handler, after validation passes:
const submissionCount = countSubmission();
console.log("II.5 Successful submission count:", submissionCount);
```

count is declared inside createSubmissionCounter, so no code outside can read or modify it directly. The inner function returned by createSubmissionCounter keeps access to count even after the outer function has finished. That combination of an inner function plus the variables it captured is a closure. Each call to countSubmission() increments the same private count.

Output

II.5 Successful submission count: 2	app.js:95
=== Recall notice submitted successfully ===	app.js:69
II.2 Form data as a JSON string:	app.js:73
{	app.js:74
"productName": "Salmon wild caught",	
"recallingFirm": "Seafood Inc",	
"submitterEmail": "seafood@example.com",	
"description": "The salmon sold by seafood company was tested with	
bacterials and the percentage of the bacterial is beyond the benchmark. ",	
"category": "Bacterial Contamination",	
"agreedToTerms": true	
}	
II.3 Destructured from the parsed object:	app.js:81
productName : Salmon wild caught	app.js:82
submitterEmail: seafood@example.com	app.js:83
II.4 Parsed object with submissionDate added:	app.js:90
▼ {productName: 'Salmon wild caught', recallingFirm: 'Seafood Inc', submitterEmail: 'seafood@example.com', description: 'The salmon sold by seafood company was tested with...centage of the bacterial is beyond the benchmark. ', category: 'Bacterial Contamination', ...} ⓘ	app.js:91
agreedToTerms: true	
category: "Bacterial Contamination"	
description: "The salmon sold by seafood company was tested with bacter	
productName: "Salmon wild caught"	
recallingFirm: "Seafood Inc"	
submissionDate: "2026-08-31T17:47:24.950Z"	
submitterEmail: "seafood@example.com"	
▶ [[Prototype]]: Object	
II.5 Successful submission count: 3	app.js:95

Deployment

Docker — local build and run

The Dockerfile lives at `code/Dockerfile`. The build context is `code/web_application/`, so only `index.html` and `app.js` are copied into the image; the Python files under `code/` are excluded.

Commands

```
cd ~/Documents/data260-9275

docker build -f code/Dockerfile -t recall-notice-app code/web_application

docker run -d -p 8275:80 --name recall-notice recall-notice-app
```

Nginx listens on port 80 inside the container. The `-p 8275:80` flag maps host port 8275 to container port 80, so the application is reached at `http://localhost:8275` — `PORT_BASE` from Section 0.

Dockerfile

```
FROM nginx:alpine

RUN rm -rf /usr/share/nginx/html/*

COPY . /usr/share/nginx/html/

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Output

The screenshot shows a web browser window with two tabs, both labeled 'HW1-Xu Wang'. The address bar shows 'localhost:8275'. The page displays a 'Recall Notice' form. The form has the following sections:

- Product Name ***: A text input field with placeholder text 'Recalled product name, e.g. Creamy Peanut Butter, 16 oz jar'.
- Recalling Firm ***: A text input field with placeholder text 'Company issuing the recall, e.g. Sunrise Valley Foods, Inc.'.
- Submitter Email ***: A text input field with placeholder text 'Contact email for follow-up, e.g. quality@example.com'.
- Recall Details ***: A text area with placeholder text 'Describe the problem, affected lot codes and best-by dates, distribution area, and what consumers should do'.
- Reason for Recall ***: A dropdown menu with the option 'Select a reason'.
- ☐ I agree to the terms and conditions.
- Submit Recall Notice**: A red button.

AWS ECS — one task running the same image

The image pushed to ECR was built with `--platform linux/amd64`. The Mac is Apple silicon, so a default build produces an arm64 image and the Fargate task fails with exec format error.

Commands

```
aws ecr create-repository --repository-name recall-notice-app --region us-east-1

aws ecr get-login-password --region us-east-1 | docker login --username AWS \
  --password-stdin 457145067527.dkr.ecr.us-east-1.amazonaws.com

docker build --platform linux/amd64 -f code/Dockerfile \
  -t recall-notice-app:amd64 code/web_application

docker tag recall-notice-app:amd64 \
  457145067527.dkr.ecr.us-east-1.amazonaws.com/recall-notice-app:latest

docker push 457145067527.dkr.ecr.us-east-1.amazonaws.com/recall-notice-app:latest
```

Cluster	data260-hw1-cluster
Service	recall-notice-service
Task definition	recall-notice-task (Fargate, Linux/X86_64, 0.25 vCPU, 0.5 GB)
Desired tasks	1
Container port	80 (nginx); security group allows inbound HTTP from 0.0.0.0/0
Public IP	54.205.107.97
Region	us-east-1

One problem worth recording: the first attempt to create the cluster failed with “Unable to assume the service linked role.” The AWS account was new and the console was left in the Ohio region while the image had been pushed to N. Virginia. Switching the console to us-east-1 and confirming `AWSServiceRoleForECS` existed resolved it.

The service and cluster were deleted immediately after the screenshot was taken, so no Fargate charges continued to accrue.

The screenshot shows the AWS Management Console for Amazon Elastic Container Service (ECS) in the us-east-1 region. The breadcrumb navigation indicates the path: Clusters > data260-hw1-cluster > Services. The left sidebar contains navigation links for Favorites and recents, Express Mode, Clusters, Namespaces, Task definitions, Daemon task definitions, Account settings, AWS Resource Explorer, AWS Batch, Amazon ECR, and Repositories. The main content area displays the 'data260-hw1-cluster' overview, including its ARN, status (Active), CloudWatch monitoring (Default), and action logs (Turned off). It also shows 1 active service and 1 running task. Below the overview, there are tabs for Services, Tasks, Daemons, Infrastructure, Metrics, Scheduled tasks, and Configuration. The 'Services' tab is active, showing a list of services with filters for launch type and scheduling strategy.

Output

The screenshot shows a web form titled 'Recall Notice'. The form is set against a light gray background. It contains several input fields and a submit button. The fields are: 'Product Name' (with a placeholder example), 'Recalling Firm' (with a placeholder example), 'Submitter Email' (with a placeholder example), 'Recall Details' (a larger text area with a placeholder), and 'Reason for Recall' (a dropdown menu). Below the dropdown is a checkbox for 'I agree to the terms and conditions.' and a red 'Submit Recall Notice' button.

Part 2 — Agentic AI

Files: code/agents_demo.py (pipeline), src/model_client.py (model adapter)

Command used

```
cd ~/Documents/data260-9275/code
python agents_demo.py
```

How the pipeline works, in my own words

The input is a title and a content block. Three stages run in sequence.

```
input (title + content)
|
Planner    model call 1: propose 3 tags and a summary
|
Reviewer   model call 2: check the Planner's work against the same source text
|
Finalizer  plain Python, no model call: enforce the output contract
|
Publish JSON
```

Planner

The Planner receives the title and content and nothing else. It is told to produce exactly three short tags and a summary of at most 25 words, and that every tag must use words that appear in or are directly stated by the source text. It returns JSON.

Reviewer

The Reviewer receives the same title and content plus the Planner's proposal. It checks four things: that there are exactly three tags with no near-duplicates, that each tag is grounded in the source rather than in outside knowledge, that each tag is specific rather than a generic category word, and that the summary is within the word limit and supported by the content. It returns corrected tags and a summary, a boolean saying whether it changed anything, and a one-sentence note explaining why.

Finalizer

The Finalizer is deterministic Python and makes no model call. It lowercases and deduplicates the tags, pads from the Planner's list if the Reviewer returned fewer than three, and truncates the summary to 25 words. I did it this way because a language model does not guarantee it will obey a count or a length limit, and the assignment requires exactly three tags and at most 25 words. Putting the hard contract in code means the output is always valid regardless of what the model returns. The model supplies judgement; the code supplies the guarantee.

Strict JSON output

Both agents are given a JSON schema that Ollama enforces server-side through its format parameter, rather than being asked in prose to return JSON.

```
PLANNER_SCHEMA = {
    "type": "object",
    "properties": {
        "tags": {
            "type": "array",
            "items": {"type": "string"},
            "minItems": 3,
            "maxItems": 3,
        },
        "summary": {"type": "string"},
        "rationale": {"type": "string"},
    },
    "required": ["tags", "summary"],
}
```

Two defensive layers back this up, because a local model still returns unexpected text sometimes. `strip_model_artifacts` removes qwen3's `<think>...</think>` reasoning block and any markdown code fence; `extract_json_object` then scans for the first balanced pair of braces if a plain `json.loads` fails.

```
THINK_BLOCK = re.compile(r"<think>.*?</think>", re.DOTALL | re.IGNORECASE)
CODE_FENCE = re.compile(r"```(?:json)?\s*(.*?)```", re.DOTALL)

def strip_model_artifacts(text: str) -> str:
    text = THINK_BLOCK.sub(" ", str(text))
    fenced = CODE_FENCE.search(text)
    if fenced:
        text = fenced.group(1)
    return text.strip()
```

Requirement 4: the domain is not hardcoded

Neither `agents_demo.py` nor `model_client.py` contains the words food, grocery, recall, allergen, or any other domain term. The prompts refer only to TITLE and CONTENT. The same script runs unchanged on any of the eight assigned domains.

To make that claim checkable rather than merely asserted, the Finalizer prints a grounding report: for each final tag, whether every word in it also occurs in the input text. Comparison uses a crude language-general stemmer so that word forms match.

```

SUFFIXES = ("ing", "ies", "es", "ed", "s")

def stem(word: str) -> str:
    for suffix in SUFFIXES:
        if word.endswith(suffix) and len(word) - len(suffix) >= 3:
            base = word[:-len(suffix)]
            if suffix == "ies":
                base += "y"
            return base
    return word

def grounding_report(tags, title, content):
    source_words = {stem(w) for w in WORD.findall(f"{title} {content}".lower())}
    report = {}
    for tag in tags:
        tag_words = [stem(w) for w in WORD.findall(tag)]
        report[tag] = bool(tag_words) and all(w in source_words for w in tag_words)
    return report

```

The stemmer was added after the first run exposed a defect in the original exact-match version. That episode is written up in [AI_USE.md](#).

Console output

```

[(data260) Sunny@MacBook-Pro-7 code % python agents_demo.py]
=====
PLANNER (agent 1)
=====
{
  "tags": [
    "peanut butter",
    "milk allergy",
    "product recall"
  ],
  "summary": "Sunrise Valley Foods recalls peanut butter jars due to potential milk contamination posing allergy risks."
}
[tokens in=293 out=43 total=336 latency=47224 ms]

=====
REVIEWER (agent 2)
=====
{
  "tags": [
    "peanut butter",
    "milk contamination",
    "product recall"
  ],
  "summary": "Sunrise Valley Foods recalls peanut butter jars due to potential milk contamination posing allergy risks.",
  "changed": true,
  "notes": "Changed 'milk allergy' to 'milk contamination' for specificity and to better reflect the content."
}
[tokens in=348 out=75 total=423 latency=25414 ms]

=====
FINALIZER (deterministic, no model call)
=====
reviewer changed the planner's work : True
summary word count : 15 / 25
tag grounded in the input text?
  yes peanut butter
  no milk contamination
  yes product recall

=====
PUBLISH
=====
{
  "tags": [
    "peanut butter",
    "milk contamination",
    "product recall"
  ],
  "summary": "Sunrise Valley Foods recalls peanut butter jars due to potential milk contamination posing allergy risks."
}

total latency: 72654 ms

```

Publish JSON produced by the run recorded above

```
{
  "tags": [
    "peanut butter",
    "milk contamination",
    "product recall"
  ],
  "summary": "Sunrise Valley Foods recalls peanut butter jars due to potential milk contamination posing allergy risks."
}
```

Short answers**Q1. The three final tags obtained**

peanut butter, milk contamination, product recall

Q2. The final summary (25 words or fewer)

"Sunrise Valley Foods recalls peanut butter jars due to potential milk contamination posing allergy risks." (15 words)

Q3. Did the Reviewer agent change anything?

Yes. The Planner proposed peanut butter, milk allergy, product recall. The Reviewer replaced milk allergy with milk contamination and recorded the reason in its notes field: "Changed 'milk allergy' to 'milk contamination' for specificity and to better reflect the content." Across the 40 runs of Part 3 the Reviewer changed the Planner's output in 37 of them, so this is its normal behaviour rather than a one-off.

Part 3 — Measuring Non-Determinism

Files: code/nondeterminism_runner.py, reports/hw01/METRICS.md, reports/hw01/raw/

Setup

Frozen input	reports/hw01/cases/nondeterminism_input.json (89-word recall notice)
Pipeline	code/agents_demo.py, Planner → Reviewer → Finalizer
Model	qwen3:8b via Ollama, context length 40,960
Hardware	MacBook Pro, 8 GB RAM
Runs	20 at temperature 0.7, 20 at temperature 0.0
Run window	2026-08-30, 21:51–22:01 PDT
Raw data	reports/hw01/raw/nondeterminism_runs.jsonl and .csv
Console log	reports/hw01/RUN_LOG.txt

Command used

```
python -u nondeterminism_runner.py 2>&1 | tee ../reports/hw01/RUN_LOG.txt
```

Tag sets are compared without regard to order, so a reordering of the same three tags counts as one set rather than two. Each run is appended to the JSONL file as soon as it finishes, so an interruption loses at most one run and a re-run resumes rather than starting over.

Required results

Metric	Temp 0.7	Temp 0.0
Distinct tag sets	2	1
Tags in all 20 runs	2	3
Tags in exactly 1 run	0	0
Latency p50 / p95 / p99 (ms)	12,122 / 15,876 / 18,586	12,378 / 30,359 / 36,294

Additional measures from the same 40 runs

Metric	Temp 0.7	Temp 0.0
Distinct summaries	9	1
Runs where the Reviewer changed the output	17 / 20	20 / 20
Distinct tags seen overall	4	3

Tag frequency

Tag	Temp 0.7 (of 20)	Temp 0.0 (of 20)
peanut butter	20	20
product recall	20	20
milk contamination	18	20
milk allergy	2	0

```

=====
PART 3 RESULTS
=====
Metric                Temp 0.7                Temp 0.0
-----
Runs completed         20                     20
Distinct tag sets      2                      1
Tags in all runs       2                      3
Tags in exactly 1 run  0                      0
Distinct summaries     9                      1
Reviewer changed (count) 17                     20
Latency p50 (ms)       12121.5                12377.6
Latency p95 (ms)       15876.4                30359.4
Latency p99 (ms)       18586.1                36294.4

--- Temperature 0.7 ---
tags present in all 20 runs : ['peanut butter', 'product recall']
tags present in exactly 1 run : none
tag frequency (tag: runs out of 20)
| 20 peanut butter
| 20 product recall
| 18 milk contamination
| 2 milk allergy

--- Temperature 0.0 ---
tags present in all 20 runs : ['milk contamination', 'peanut butter', 'product recall']
tags present in exactly 1 run : none
tag frequency (tag: runs out of 20)
| 20 peanut butter
| 20 milk contamination
| 20 product recall

raw per-run JSONL : reports/hw01/raw/nondeterminism_runs.jsonl
raw per-run CSV   : reports/hw01/raw/nondeterminism_runs.csv
metrics JSON      : reports/hw01/raw/nondeterminism_metrics.json
model: qwen3:8b   temperature: 0.0
system prompt: AGENT.md (1212 chars)

```

Observations

1. Temperature 0.0 produced no variation at all in these runs.

One tag set and one summary across 20 runs. This does not prove the pipeline is deterministic in general. Floating-point accumulation order on the GPU and server-side batching can still introduce differences, and 20 runs on a single input is a small sample. It shows only that no variation surfaced under these conditions.

2. Tags were far more stable than summaries.

At temperature 0.7 there were 2 distinct tag sets but 9 distinct summaries, from the same runs at the same temperature. The tag field is short, capped at three items, and drawn from words in the source; the summary is free prose with many equally valid phrasings. What made the output reproducible was constraining its shape, not lowering the temperature.

3. The single tag that varied is the safety-relevant one.

Two of the three tags were identical in all 40 runs. The one that moved was the hazard tag: milk contamination in 18 runs, milk allergy in 2. Both describe the same underlying hazard, but they are different strings, and any downstream system that routes on the tag would treat them as different categories.

4. Temperature did not change per-run cost; machine state did.

The p50 latencies are within 2% of each other, 12.1 s against 12.4 s, which is what should be expected: sampling temperature changes which token is chosen, not how many tokens are produced or how fast

each one is produced. The much heavier tail at temperature 0.0 (p95 of 30.4 s against 15.9 s) is not a temperature effect. In RUN_LOG.txt the slow runs form a contiguous block, runs 10 through 15, at 20 to 36 seconds, while every run before and after is 10 to 13 seconds. A contiguous block is the signature of machine state, in this case memory pressure on an 8 GB machine, not of sampling behaviour. Reading this as “temperature 0.0 is slower” would be a misreading of my own data.

5. The Reviewer agent does real work.

It altered the Planner's output in 37 of 40 runs, almost always replacing milk allergy with milk contamination. The three runs where it made no change are the runs where the Planner had already produced the tag the Reviewer preferred.

Required analysis

What two users sending identical input might see

At temperature 0.7, two users submitting this same recall notice would receive the same three tags about 90% of the time (18 of 20 runs) and different tags about 10% of the time: one sees milk contamination, the other sees milk allergy. The summaries differ far more often, with 9 distinct wordings in 20 runs, so two users are more likely than not to read differently-phrased summaries of the same facts. At temperature 0.0 both users saw identical output in every run.

One case where run-to-run variation is acceptable

Summary wording shown to a human reader. All 9 distinct summaries carried the same facts: the product, the undeclared milk, the allergy risk, and the recall. A person reading any one of them learns the same thing and takes the same action. The same holds for using tags as search facets or to surface related notices; a user who sees milk allergy instead of milk contamination still finds relevant material. Nobody is harmed by the difference.

One case where it is not acceptable

Routing allergen alerts by tag. If the tag decides which subscribers are notified, for example everyone who has registered a milk allergy, then the 10% of runs producing milk allergy instead of milk contamination reach a different audience. Under the FDA classification a Class I recall is one with a reasonable probability of serious adverse health consequences or death, and undeclared milk in a peanut butter product is exactly that case. A one-in-ten chance of routing a life-safety alert to the wrong list is a defect, not tolerable variance.

The fix is not simply setting temperature to 0. It is taking the safety-critical field out of free generation entirely. The submission form in Part 1 makes the user choose category from four fixed values rather than letting a model produce that string. Free-text tags stay useful for search and discovery, where the variation measured above does no harm.

Part 4 — Model Client and Token Accounting

Files: src/model_client.py (adapter), code/hw1_client.py (CLI), AGENT.md (system prompt)

The adapter

src/model_client.py exposes one method, complete(messages, tools=None). Every model call in the repository goes through it, including both agents in Part 2. Swapping the backend means editing this one file.

It calls the Ollama Python client directly rather than a framework wrapper, because the Ollama response carries prompt_eval_count and eval_count, which are the per-turn input and output token counts this part must report. A wrapper would hide them.

```
def complete(self, messages, tools=None, temperature=None,
              response_format=None, think=False):
    temp = self.temperature if temperature is None else temperature
    kwargs = {"model": self.model, "messages": messages,
              "options": {"temperature": temp}}
    if tools:
        kwargs["tools"] = tools
    if response_format is not None:
        kwargs["format"] = response_format

    started = time.perf_counter()
    response = self._client.chat(think=think, **kwargs)
    latency_ms = (time.perf_counter() - started) * 1000.0

    payload = dict(response)
    input_tokens = int(payload.get("prompt_eval_count") or 0)
    output_tokens = int(payload.get("eval_count") or 0)

    self.turn_count += 1
    self.cumulative_input_tokens += input_tokens
    self.cumulative_output_tokens += output_tokens
    ...
```

think defaults to False. ollama show qwen3:8b lists thinking among the model's capabilities, so without this the reply would be prefixed by a <think> block.

AGENT.md and verifying the model follows it

AGENT.md is read from disk at startup and used verbatim as the system prompt. Changing the assistant's behaviour needs no code change. It demands bullet-only replies:

```
## Output format - strict

Reply with bullet points only.

- Every line of your reply MUST begin with `- `.
- No preamble, no greeting, no closing sentence, no summary paragraph.
- No headings, no numbered lists, no code fences, no tables.
- One finding per bullet. At most 6 bullets per reply.
- Each bullet is one sentence, at most 25 words.
- If the code has no defects, reply with the single bullet `- No defects found.`
```

Asking for a format is not the same as getting it, so `hw1_client.py` measures compliance after every turn instead of assuming it.

```
def check_bullet_only(reply):
    lines = [ln.strip() for ln in reply.strip().splitlines() if ln.strip()]
    violations = []
    non_bullet = [ln for ln in lines if not ln.startswith("- ", "* ")]
    if non_bullet:
        violations.append(f"{len(non_bullet)} line(s) do not start with a bullet")
    if len(lines) > 6:
        violations.append(f"{len(lines)} bullets, limit is 6")
    if "```" in reply:
        violations.append("contains a code fence")
    ...
```

Result: the model complied on turns 1 to 4 and violated the contract on turn 5.

```
[turn 4] input tokens = 774  output tokens = 35  total tokens = 809  (5480 ms)
[AGENT.md bullet-only check] PASS (1 bullets)

[turn 5] input tokens = 834  output tokens = 244  total tokens = 1078  (24959 ms)
[AGENT.md bullet-only check] FAIL -> 13 bullets, limit is 6
```

Turn 5 asked the model to list every distinct defect it had reported across the conversation. That request conflicted with the six-bullet cap, and the model chose to satisfy the request. It also ignored the instruction not to repeat itself, reproducing all thirteen earlier bullets verbatim rather than deduplicating them. A system prompt is a strong prior on behaviour, not an enforced constraint. This is why the check exists in code.

Five-turn conversation

Command used

```
python -u hw1_client.py --script 2>&1 | tee -a ../reports/hw01/RUN_LOG.txt
```

The five turns are deliberately ordered. Turns 1 to 3 are independent code reviews. Turn 4 asks which of the three functions shown so far is most fragile, and turn 5 asks for every distinct defect reported across the whole conversation. Neither of the last two can be answered from the question alone; both require the earlier turns to be present in the request. That is the experimental evidence for the first conceptual question below.

Per-turn token accounting

Turn	Input tokens	Output tokens	Total	Latency (ms)	AGENT.md check
1	344	25	369	17,715	PASS
2	463	101	564	12,289	PASS
3	622	118	740	16,385	PASS
4	774	35	809	5,480	PASS
5	834	244	1,078	24,959	FAIL
Total	3,037	523	3,560	—	4 of 5

/stats after turn 3

```
turn count           : 3
cumulative input tokens : 1429
cumulative output tokens : 244
cumulative total tokens : 1673
conversation messages  : 7
serialized history length (chars) : 3359
serialized history length (bytes) : 3361
```

```
=====  
/stats  after turn 3  
=====
```

```
turn count           : 3
cumulative input tokens : 1429
cumulative output tokens : 244
cumulative total tokens : 1673
conversation messages  : 7
serialized history length (chars) : 3359
serialized history length (bytes) : 3361  
=====
```

/stats after turn 5

```

turn count           : 5
cumulative input tokens : 3037
cumulative output tokens : 523
cumulative total tokens : 3560
conversation messages  : 11
serialized history length (chars) : 5030
serialized history length (bytes) : 5032

```

```

=====
/stats  after turn 5
=====

```

```

turn count           : 5
cumulative input tokens : 3037
cumulative output tokens : 523
cumulative total tokens : 3560
conversation messages  : 11
serialized history length (chars) : 5030
serialized history length (bytes) : 5032
=====

```

```

=====
SESSION TOTALS
=====

```

```

turns                : 5
cumulative input tokens : 3037
cumulative output tokens : 523
cumulative total tokens : 3560
serialized history length (chars) : 5030
=====

```

/stats does not alter the history. It calls `history_length()`, which serializes `self.history` to measure it and writes nothing back. In interactive mode the command is intercepted before any message is appended, so the counters and the conversation both stay clean.

Two incorrect reviews the model produced

Both were caught by reading the reply against the code rather than accepting it.

Turn 1. The model reported:

- The ``count`` variable is not declared with ``const`` or ``let``, leading to a potential global variable.

The submitted code declares it on the second line: `let count = 0;`. The defect does not exist.

Turn 3. The model reported:

- The function incorrectly returns ``True`` as a boolean but uses division to achieve it, which is unnecessary and confusing.

`clamp_summary` contains no division of any kind. The claim is fabricated.

Both replies were fluent, correctly formatted, and passed the bullet-only check, which is the point: format compliance says nothing about factual correctness. These are recorded in `AI_USE.md`.

Conceptual questions

Why is prior conversation context resent with every turn?

Because the model holds no state between calls. Each request is independent and the server keeps nothing from the previous one, so anything the model needs must be inside the request.

Turn 4 shows this directly. The question was 108 characters, roughly 25 tokens, but the turn billed 774 input tokens. The other ~750 were the earlier conversation being replayed. That replay is what let the model answer “of the three functions I have shown you so far” correctly; with only the question it would have had no idea which three functions were meant.

This is the same statelessness REST requires of a web service: the server does not remember the client, so the client carries the context.

How is a system prompt different from a user message?

Position and persistence: the system prompt is message index 0 and is resent unchanged every turn, while user messages accumulate behind it. Purpose: it sets standing rules for how to answer rather than asking a question; AGENT.md never requests anything, it constrains the shape of every reply. Cost: its 1,212 characters were billed in all five turns, so a system prompt is a fixed tax on every request rather than a one-time setup cost.

It is not an enforcement mechanism. Turn 5 produced 13 bullets against a cap of 6, because the user's request conflicted with the cap and the model chose the request.

Why do input tokens grow over a conversation?

Every turn sends the system prompt, every previous user message, every previous assistant reply, and the new message. Nothing is removed, so the request lengthens each time: 344, 463, 622, 774, 834. Each increase tracks what was added since the previous turn; input rose 159 tokens from turn 2 to turn 3, and turn 2's reply was 101 tokens plus the new question.

The cumulative effect is lopsided: 3,037 input tokens against 523 output tokens, so 85% of everything processed was input and most of that was history being re-read. Because turn n carries everything before it, total tokens over a conversation grow roughly with the square of the turn count rather than linearly.

What eventually limits that growth?

The context window: the maximum number of tokens the model will accept in one request. Once history plus the new message exceeds it, the request either fails or the oldest messages are silently dropped and the model forgets the start of the conversation without saying so.

For this setup ollama show qwen3:8b reports a context length of 40,960 tokens. Input reached 834 by turn 5 and grew by roughly 120 to 150 tokens per turn, that growth being the previous reply plus the new question, so the window fills somewhere around turn 300. The effective ceiling can be lower: the running server's num_ctx may be set below what the model supports, and the smaller number applies.

Before that hard ceiling, latency bites first, and this session shows it tracks output rather than input. Turn 4 read 774 input tokens and finished in 5.5 s because it wrote only 35 tokens; turn 5 read a similar 834 and took 25.0 s because it wrote 244. Input is processed in one parallel prefill pass while output is generated one token at a time. Input is therefore cheap per token but grows without bound, which is what later techniques address: trimming or summarising old turns, retrieving only the relevant history instead of all of it, and caching the unchanged prefix so the same prefill is not paid for twice.